YALNIZ MÜƏLLİM ÜÇÜN · ARXA MONİTOR

Dərs 14 — DRF və ilk API view-lar addım-addım

REST, JSON, serializer
api app · ModelSerializer · @api_view · Postman
Slaydlar: lesson-14.pdf (35)

Başlanğıc — Dərs 13-ün sonundakı layihə. Hər addımda faylın həmin andakı vəziyyəti göstərilir. Yaşıl sətirlər — bu addımda **yeni yazdığın** sətirlərdir; ~~qırmızı~~ — silinən/əvəz olunan sətir; ağ sətirlər artıq fayldadır, `...` — dəyişməyən hissə. Qara bloklar — terminal. Xəta mətnləri real yoxlanılıb. Sağ küncdə — təxmini vaxt və slayd nömrəsi.

0 · Dərsdən əvvəl (evdə, 10 dəq)

1 Hazırlıq

dərsdən əvvəl

1. Tələbələrə **bir gün əvvəl** yaz: “Postman yükləyin” (`postman.com/downloads`). uv-ni Dərs 7-dən bilirlər — dərsdə izah edilmir, yalnız `uv add` işlənir.

```
uv --version                  # uv 0.11.21 (...) — layihə artıq uv ilədir (pyproject.toml + uv.lock)
uv sync                       # Dərs 13-ün layihəsini aç: .venv + paketlər
uv run manage.py runserver    # Dərs 13-ün saytı açılmalıdır
```

2. Dərs 13-ün sonundakı layihəni aç (öz `step-blog`-un və ya `project/django/lesson-13/`-nin surəti). Bu gün **model və migration dəyişmir** — baza olduğu kimi qalır. Bazanda ən azı 5 post olsun (`loaddata demo_full`).

3. Postman-ı aç, boş `step-blog` collection-ı yarat. Editorda `blog/views.py` yan-yana açıq qalsın — “iki dəfə yaz” müqayisəsi üçün hər addımda ona işarə edəcəksən.

Qızıl qayda: HTML sayta bu gün **toxunmuruq**. Hər şey yeni `api/` qovluğuna yazılır. Tələbə `blog/views.py`-ı açıb “bunu dəyişim?” deyəndə — yox.

I hissə · Slayd + canlı kod (0:00 – 1:30)

2 Giriş — mobil tətbiq motivasiyası (kod yoxdur)

0:00–0:10 · slayd 1–4

Slayd 2: keçən dərsdən 5 sual + ev tapşırığına baxış (like, `/liked/`, `Q`). Slayd 3: brauzerdə `/post/1/` aç, sonra **Ctrl+U** (mənbə) — HTML-i göstər.

> **De:** “Bu blogun telefon tətbiqini istəyirlər. Tətbiqə bu HTML lazımdırmı? Yox — ona **məlumat** lazımdır: başlıq, müəllif, mətn. Dizaynı tətbiq özü çəkir. Biz 7 dərsdə blogu bir dəfə yazdıq — indi eyni blogu **ikinci dəfə**, API kimi yazırıq. Model, baza, admin dəyişmir. Dəyişən yalnız cavabın formasıdır: HTML yox, JSON.”

Slayd 4: bu günün planı. Vurğula: II hissə yenə əkizdir — mən postlar, sən kateqoriyalar.

3 API, REST, JSON — slayd + JsonResponse demosu

0:10–0:27 · slayd 5–12

Slayd 6–10: restoran analogiyası, HTML vs JSON, JSON tipləri, REST cədvəli, status kodları. Tez keç — hamısı Dərs 7-nin davamıdır. Slayd 11 — mikro-tapşırıq 1 (aşağıda). Slayd 12 üçün **canlı demo** — `blog/views.py`-ın sonuna müvəqqəti view:

blog/views.py (sonuna, müvəqqəti)

```
# MÜVƏQQƏTİ — DRF-siz JSON. Demo bitəndə SİLİNİR
from django.http import JsonResponse

def post_json(request, post_id):
    post = Post.objects.get(id=post_id)
    data = {
        "id": post.id,
        "title": post.title,
        "author": post.author.username,
        "created_at": post.created_at,
    }
    return JsonResponse(data)
```

blog/urls.py (müvəqqəti)

```
    path("post/<int:post_id>/json/", views.post_json),          # müvəqqəti
```

```
# brauzerdə /post/1/json/ :
{"id": 1, "title": "Python-da ilk add\u0131m", "author": "nigar", "created_at": "2026-09-01T06:00:00Z"}
```

De: “İşləyir. Amma 4 sahəni əl ilə yazdım. Post-da 10 sahə var, 5 modelimiz var — 50 sətir əl işi. Bir də tərsinə: JSON gələndə onu yoxlayıb bazaya kim yazacaq?”

⚠ **Bilə-bilə səhv et:** `return JsonResponse({"post": post})` → `TypeError: Object of type Post is not JSON serializable`. “JSON Django obyektini tanımır. Tərcüməçi lazımdır — **serializer**.”

Demo bitdi — `post_json` view-unu, URL-i və `JsonResponse` importunu **sil**. Layihədə qalmır.

4 Mikro-tapşırıq 1

0:22 · slayd 11

5 dəqiqə, kağızda — REST forması + status kodu. Cavablar: **1** `GET /api/films/` → 200 · **2** `DELETE /api/films/3/` → 204 · **3** 201 (yeni obyekt) · **4** 400 (`year` tələb edilir) · **5** 404. Soruş: “4 və 5 — kim səhv etdi?” → tətbiq (4xx). “500 nə vaxt olar?” → bizim kodda xəta.

5 DRF quraşdırma + api app

0:27–0:37 · slayd 14–15

```
uv add djangorestframework                    # bu günün paketi (slayd 14 — uv-ni xatırla)
Resolved 8 packages in 4ms
Installed 1 package in 6ms
 + djangorestframework==3.18.1
```

pyproject.toml (uv özü yazdı — biz əl vurmadıq)

```
dependencies = [
    "django>=5.2",
    "djangorestframework>=3.18.1",
    "pillow>=11",
]
```

> **De:** “Dərs 7: paket lazımdır → `uv add` . `pyproject.toml` və `uv.lock` yeniləndi — ikisi də commit-ə gedir. Vəssalam, uv haqqında bu gün bu qədər.”

```
uv run manage.py startapp api
ls api
__init__.py  admin.py  apps.py  migrations  models.py  tests.py  views.py
```

config/settings.py (INSTALLED_APPS)

```
    'blog',  # [DƏRS 7]
    'accounts',  # [DƏRS 12] qeydiyyat, giriş, profil öz app-ımız — bunu yazmasaq Django app-ı görmür
    'rest_framework',  # [DƏRS 14] Django REST Framework — uv add djangorestframework
    'api',  # [DƏRS 14] blogun API üzü: serializer + API view-lar. Modellər blog-da qalır
]
```

config/urls.py

```
    path('admin/', admin.site.urls),
    path('accounts/', include('accounts.urls')),  # [DƏRS 12]
    path('api/', include('api.urls')),  # [DƏRS 14] /api/... ünvanları API app-ına gedir — HTML blog toxunulmur
    path('', include('blog.urls')),  # [DƏRS 7] qalan bütün ünvanlar blog/urls.py-a gedir
]
```

api/urls.py — YENİ FAYL

```
from django.urls import path

from . import views

app_name = "api"

urlpatterns = [
]
```

> **De:** “Niyə ayrı app? HTML sayt `blog` -da qalır, ona toxunmuruq — sabah yenə işləməlidir. API `api` -dədir. Modellər bir yerdədir: `from blog.models import Post` . `api/models.py` boş qalacaq.”

`api/urls.py` -ı boş `urlpatterns` ilə yarat — yoxsa `runserver` `ModuleNotFoundError: No module named 'api.urls'` verir.

> **Yoxla:** `uv run manage.py runserver` xətasız qalxır. Sayt əvvəlki kimi.

6 PostSerializer

0:37–0:45 · slayd 16–17

api/serializers.py — YENİ FAYL

```python
from rest_framework import serializers

from blog.models import Post


# [DƏRS 14] müəllim yazır
class PostSerializer(serializers.ModelSerializer):
    class Meta:
        model = Post                                            # hansı model (ModelForm-dakı kimi)
        fields = [                                              # JSON-da hansı sahələr olsun
            "id", "title", "author", "category", "tags",
            "content", "image", "is_published", "views", "created_at",
        ]
        read_only_fields = ["views", "created_at"]              # yalnız oxunur — POST-da göndərilsə də, nəzərə alınmır
```

De: “`blog/forms.py`-ı aç — `PostForm`. Eyni qəlib: `class Meta`, `model`, `fields`. ModelForm HTML üçün tərcüməçi idi, ModelSerializer JSON üçün. Üç iş: obyekt → dict, JSON → obyekt + yoxlama, save.”

```
uv run manage.py shell
>>> from blog.models import Post
>>> from api.serializers import PostSerializer
>>> PostSerializer(Post.objects.get(id=1)).data
{'id': 1, 'title': 'Python-da ilk addım', 'author': 1, 'category': 1, 'tags': [1], 'content': '...', 'image': None,
'is_published': True, 'views': 0, 'created_at': '2026-09-01T10:00:00+04:00'}
>>> PostSerializer(Post.objects.all(), many=True).data
[{'id': 9, ...}, {'id': 8, ...}, ...]
```

⚠ **Bilə-bilə səhv et:** `PostSerializer(Post.objects.all()).data` (many-siz) → `AttributeError: Got AttributeError when attempting to get a value for field `title` on serializer `PostSerializer`. ... Original exception text was: 'QuerySet' object has no attribute 'title'.` “QuerySet-in `title`-ı yoxdur — siyahıdır. `many=True`.”

Diqqət çək: `author: 1`, `category: 1`, `tags: [1]` — ForeignKey/M2M **id** kimi gəlir. “Dərs 15-də adları da göstərəcəyik.” `created_at` string oldu — JSON-da datetime yoxdur.

7 Mikro-tapşırıq 2

0:45 · slayd 18

5 dəqiqə, kağızda. Cavab: `FilmSerializer(serializers.ModelSerializer)`, `fields = ["id", "title", "year", "director", "rating"]`, `read_only_fields = ["rating", "created_at"]` (created_at fields-də yoxdursa, read_only-də də lazım deyil — ikisi də qəbul olunur). 5 film → `many=True`, 1 film → many-siz. `"iki min"` → `is_valid()` False, `{"year": ["Düzgün ədəd daxil edin."]}`.

8 post_list — yalnız GET + Browsable API

0:50–0:57 · slayd 20–21

api/views.py

```python
from rest_framework.decorators import api_view
from rest_framework.response import Response

from blog.models import Post
from .serializers import PostSerializer


# [DƏRS 14] əkizi: blog/views.py → PostListView (siyahı)
@api_view(["GET"])
def post_list(request):
    posts = Post.objects.all()
    serializer = PostSerializer(posts, many=True)            # many=True → siyahı (list), tək obyekt deyil
    return Response(serializer.data)                         # .data = JSON-a hazır list[dict]
```

api/urls.py

```python
urlpatterns = [
    # [DƏRS 14]
    path("posts/", views.post_list, name="post_list"),
]
```

Brauzerdə `/api/posts/` aç → DRF-in **Browsable API** səhifəsi: JSON rəngli, yuxarıda metod, status. Sonra `/api/posts/?format=json` → xam JSON. “Mobil tətbiq bunu alır. Brauzerdə DRF bizə oxunaqlı səhifə göstərir — hədiyyədir.”

De: “`PostListView` ilə müqayisə et: obyektləri götür — eynidir. Sonra: orada template + context, burada serializer + Response. `@api_view` — decorator, Dərs 2. Adi funksiyanı API view-a çevirir: JSON oxuyur, metodu yoxlayır, xətanı JSON edir.”

⚠ **Bilə-bilə səhv et:** settings-dən `'rest_framework'`-ü şərhə al, səhifəni yenilə → `TemplateDoesNotExist: rest_framework/api.html`. “DRF-in səhifəsi app-ın içindədir. App qeydiyyatda deyil — Django tapmır.” Geri qaytar.

Yoxla: `/api/posts/` → 9 post JSON. `?format=json` işləyir.

9 post_detail — GET + 404

0:57–1:02 · slayd 22

api/views.py

```python
from django.shortcuts import get_object_or_404
from rest_framework.decorators import api_view
...

# [DƏRS 14] əkizi: post_detail
@api_view(["GET"])
def post_detail(request, post_id):
    post = get_object_or_404(Post, id=post_id)                  # tapmasa DRF 404 JSON qaytarır: {"detail": "..."}
    serializer = PostSerializer(post)                           # bir obyekt → many yazılmır
    return Response(serializer.data)
```

api/urls.py

```python
    path("posts/", views.post_list, name="post_list"),
    path("posts/<int:post_id>/", views.post_detail, name="post_detail"),
```

```
# /api/posts/1/    → 200  {"id":1,"title":"Python-da ilk addım", ...}
# /api/posts/999/  → 404  {"detail":"No Post matches the given query."}
```

De: “Dərs 9-dakı `get_object_or_404` — eyni funksiya. Fərq: DRF 404-ü HTML səhifə yox, JSON edir. Tətbiq `detail`-i oxuyub “tapılmadı” yazar.”

10 Postman — GET sorğuları

1:02–1:08 · slayd 28

Postman-ı aç: **New** → **HTTP**, `GET`, `http://127.0.0.1:8000/api/posts/`, **Send**. Göstər: status `200 OK`, vaxt, body (Pretty). Sonra `/api/posts/1/`, `/api/posts/999/` → 404.

Save → collection `step-blog`, ad "posts list". "Sabah, o biri gün yenə lazımdır — hamısını saxlayırıq."

> **De:** "Brauzer ünvan sətrindən yalnız GET edə bilir. POST, PUT, DELETE üçün alət lazımdır — Postman. Mobil tətbiq hələ yoxdur, Postman onun yerinə danışır."

> ⚠ `Could not get response` → `runserver` işləmir. `localhost` yox, `127.0.0.1` yaz (bəzi Windows-larda localhost IPv6-ya gedir).

11 post_list-ə POST — yaratmaq

1:08–1:17 · slayd 23–24, 29

`api/views.py`

```
from django.shortcuts import get_object_or_404
from rest_framework import status
from rest_framework.decorators import api_view
...

@api_view(["GET"])
# [DƏRS 14] əkizi: blog/views.py → PostListView (siyahı) + PostCreateView (yaratma) — API-də ikisi bir view-dur
@api_view(["GET", "POST"])
def post_list(request):
    posts = Post.objects.all()
    serializer = PostSerializer(posts, many=True)            # many=True → siyahı (list), tək obyekt deyil
    return Response(serializer.data)                         # .data = JSON-a hazır list[dict]
    if request.method == "GET":
        posts = Post.objects.all()
        serializer = PostSerializer(posts, many=True)            # many=True → siyahı (list), tək obyekt deyil
        return Response(serializer.data)                         # .data = JSON-a hazır list[dict]

    serializer = PostSerializer(data=request.data)               # gələn JSON-u serializer-ə ver (form =
PostForm(request.POST) kimi)
    if serializer.is_valid():                                    # bütün yoxlamalar keçdi?
        serializer.save()                                        # bazaya yaz (ModelForm.save() kimi)
        return Response(serializer.data, status=status.HTTP_201_CREATED)        # 201 = yaradıldı
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)      # 400 = sən səhv göndərdin
```

> **De:** "`blog/views.py` → `PostCreateView` / Dərs 11 `post_create`: form = PostForm(request.POST) → is_valid → save → redirect. Burada: serializer(data=request.data) → is_valid → save → Response 201. Eyni 4 addım. `request.data` — gələn JSON, `request.POST`-un API variantı."

Postman: `POST /api/posts/`, **Body** → **raw** → **JSON**: `{"title": "Postman-dan post", "author": 1, "category": 1, "tags": [1], "content": "İlk API postum."}` → **201**, cavabda `id`. Brauzerdə `/post/10/` aç — sayt da görür. "Eyni baza." Sonra `{"title": "x"}` → **400**:

```
{"author":["Bu sahə tələb edilir."],"category":["Bu sahə tələb edilir."],"content":["Bu sahə tələb edilir."]}
```

> ⚠ **Bilə-bilə səhv et:** `is_valid()`-i çıxar, birbaşa `save()` → `AssertionError: You must call `.is_valid()` before calling `.save()`.` "Dərs 11-dəki qayda: yoxlamadan yazma. DRF bunu məcbur edir."

> ⚠ Postman-da Body **form-data** qalıbsa və `tags: [1]` göndərilirsə — 400. Həmişə **raw → JSON**.

Diqqət çək: `"author": 1` — tətbiq istədiyi adamın adından post yaza bilər. "Bu, Dərs 11-dəki vəziyyətdir: author formda idi. Dərs 16-da token gələcək — author token-dən."

12 post_detail-ə PUT + DELETE

1:17–1:24 · slayd 25, 29

`api/views.py`

```
@api_view(["GET"])
# [DƏRS 14] əkizi: post_detail + PostUpdateView + PostDeleteView — API-də üçü bir view-dur
@api_view(["GET", "PUT", "DELETE"])
def post_detail(request, post_id):
    post = get_object_or_404(Post, id=post_id)                  # tapmasa DRF 404 JSON qaytarır: {"detail": "..."}
    serializer = PostSerializer(post)                           # bir obyekt → many yazılmır
    return Response(serializer.data)

    if request.method == "GET":
        serializer = PostSerializer(post)                       # bir obyekt → many yazılmır
        return Response(serializer.data)

    if request.method == "PUT":
        serializer = PostSerializer(post, data=request.data)    # instance + data → yenilə (PostForm(request.POST,
instance=post) kimi)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)                    # 200 (default)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    post.delete()                                               # DELETE
    return Response(status=status.HTTP_204_NO_CONTENT)          # 204 = oldu, amma göndərməyə məzmun yoxdur
```

Postman: `PUT /api/posts/10/` + tam JSON (başlığı dəyiş) → 200. `DELETE /api/posts/10/` → 204, body boş. `GET /api/posts/10/` → 404. Hamısını collection-a saxla.

⚠ **Göstər:** Postman-da `PATCH /api/posts/1/` → `405` `{"detail":"\"PATCH\" yöntəminə icazə verilmir."}`. “Kod yazmadıq — `@api_view` siyahıya baxır. DRF azərbaycanca danışır: `LANGUAGE_CODE = 'az'`.”

De: “PUT = Dərs 11-dəki `PostForm(request.POST, instance=post)`. `instance` + `data` → dəyiş. DELETE — `post.delete()`, Dərs 11-dəki kimi, amma təsdiq səhifəsi yoxdur: tətbiq özü soruşar.”

Yoxla: Postman collection-da 6 sorğu: GET list, GET detail, GET 404, POST, PUT, DELETE. Hamısı gözlənilən status qaytarır.

13 Mikro-tapşırıq 3

1:24 · slayd 26

5 dəqiqə. 3 səhv: **1)** `JanrSerializer(janrlar)` — `many=True` yoxdur → AttributeError 'QuerySet' object has no attribute · **2)** `is_valid()`-dən əvvəl `save()` → AssertionError · **3)** `@api_view(["GET"])` — POST siyahıda yoxdur → 405, POST koduna heç çatmır. Gizli 4-cü: uğurlu POST `201` qaytarmalıdır, burada `200`.

14 I hissənin sonunda fayllar

1:28 · yoxlama · slayd 30

api/views.py (importlar)

```
from django.shortcuts import get_object_or_404
from rest_framework import status
from rest_framework.decorators import api_view
from rest_framework.response import Response

from blog.models import Post
from .serializers import PostSerializer
```

api/urls.py

```
urlpatterns = [
    # [DƏRS 14]
    path("posts/", views.post_list, name="post_list"),
    path("posts/<int:post_id>/", views.post_detail, name="post_detail"),
]
```

> **Yoxla:** `blog/` -da heç nə dəyişməyib (müvəqqəti `post_json` silinib). `api/serializers.py` — 1 class, `api/views.py` — 2 funksiya. `pyproject.toml` + `uv.lock` commit-ə hazırdır. Slayd 30 — xətalar cədvəli, tez keç.

II hissə · Praktika — 1 tapşırıq (1:40 – 2:20) + "İndi mən yazıram" (2:20 – 2:40)

Yeni heç nə izah etmirsən. Tək tapşırıq: "`post_list` -in əkizi — kopyala, `Post` → `Category` ." ~40 dəqiqə; 20-ci dəqiqədə ekranda checkpoint (`GET /api/categories/` kimdə açılır?), sonda Postman-da birlikdə POST. Sonra `category_detail` -i **sən yazırsan** — tələbə baxır, istəyən yanaşı yazır (slayd 34).

Nə	Tələbə nə edir	Checkpoint — Postman-da yoxla
Praktika · ~40 dəq slayd 33	`CategorySerializer` (id, name, slug, description) · `category_list` GET+POST · `path("categories/", ...)` · collection · commit	`GET /api/categories/` → 3 kateqoriya · POST `{"name":"İdman","slug":"idman"}` → 201 · POST `{"name":"x"}` → 400 slug
Mən yazıram · ~20 dəq slayd 34	`category_detail` GET/PUT/DELETE · `path("categories/<int:category_id>/", ...)` · push	`/api/categories/1/` → 200 · `/99/` → 404 · PUT → 200 · DELETE boş → 204 · DELETE postlu → 500 ProtectedError (normaldır)

15 Praktikanın həlli — yoxlamaq üçün

tələbəyə göstərmə

api/serializers.py

```
from rest_framework import serializers

~~from blog.models import Post~~
from blog.models import Post, Category
...

# [PRAKTİKA 14] — PostSerializer-in əkizi
class CategorySerializer(serializers.ModelSerializer):
    class Meta:
        model = Category
        fields = ["id", "name", "slug", "description"]
```

api/views.py

```
~~from blog.models import Post~~
~~from .serializers import PostSerializer~~
```

```python
from blog.models import Post, Category
from .serializers import PostSerializer, CategorySerializer
...

# [PRAKTİKA 14] — post_list-in əkizi (tələbə yazır)
@api_view(["GET", "POST"])
def category_list(request):
    if request.method == "GET":
        categories = Category.objects.all()
        serializer = CategorySerializer(categories, many=True)
        return Response(serializer.data)

    serializer = CategorySerializer(data=request.data)
    if serializer.is_valid():
        serializer.save()
        return Response(serializer.data, status=status.HTTP_201_CREATED)
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)
```

api/urls.py

```python
    path("posts/<int:post_id>/", views.post_detail, name="post_detail"),

    # [PRAKTİKA 14] tələbə · [DƏRS 14] category_detail-i praktikadan sonra müəllim
    path("categories/", views.category_list, name="category_list"),
```

Tez bitirənə: “Postman-da POST-u `description` ilə də göndər” və ya ev tapşırığının 1-ci bəndi (Tag). `category_detail`-i **özü yazmasın** — birlikdə yazacaqsınız.

16 İndi mən yazıram — category_detail

2:20–2:40 · slayd 34

Praktika bitdi, hamı ekrana baxır. Sən `category_detail`-i canlı yazırsan — "`post_detail`-in əkizi, 3-cü dəfə eyni qəlib". Tələbə istəsə yanaşı yazır, istəsə baxır — evdə teqlər üçün 4-cü dəfə özü yazacaq.

api/views.py

```
# [DƏRS 14] praktikadan SONRA müəllim yazır — post_detail-in əkizi ("qalanını mən yazıram, siz baxırsınız")
@api_view(["GET", "PUT", "DELETE"])
def category_detail(request, category_id):
    category = get_object_or_404(Category, id=category_id)

    if request.method == "GET":
        serializer = CategorySerializer(category)
        return Response(serializer.data)

    if request.method == "PUT":
        serializer = CategorySerializer(category, data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    category.delete()                                    # postu olan kateqoriya → on_delete=PROTECT → 500 (Dərs 10)
    return Response(status=status.HTTP_204_NO_CONTENT)
```

api/urls.py

```
    path("categories/", views.category_list, name="category_list"),
    path("categories/<int:category_id>/", views.category_detail, name="category_detail"),
```

Postman-da birlikdə: `GET /api/categories/1/` → 200 · `/99/` → 404 · `PUT /api/categories/4/` + `{"name": "Futbol", "slug": "futbol"}` → 200 · `DELETE /api/categories/4/` (praktikada yaradılan boş kateqoriya) → 204.

⚠ **Göstər:** `DELETE /api/categories/1/` (postları var) → `500` `ProtectedError`. "Dərs 10: `on_delete=PROTECT` — baza özünü qoruyur, bu düzgündür. Tətbiq 500 görəndə "əvvəl postları sil" deyər." Try/except yazma — söhbət üçün bəsdir.

De: "Bu gün eyni qəlibi 3 dəfə gördünüz: post, kateqoriya siyahısı, kateqoriya detalı. Evdə teq üçün 4-cü dəfə — kopyala, adı dəyiş, 25 dəqiqə."

Yoxla: `api/views.py`-da 4 funksiya: `post_list`, `post_detail`, `category_list`, `category_detail`. Push edildi.

Praktikada görəcəyin xəta	Səbəb
`NameError: name 'Category' is not defined`	import sətri yenilənməyib: `from blog.models import Post, Category`
`NameError: name 'CategorySerializer' is not defined`	`views.py`-da serializer importu
`'QuerySet' object has no attribute 'name'`	`many=True` unudulub
POST → `405`	`@api_view(["GET"])` — POST siyahıda yoxdur
POST → `400 {"slug": [...]}` amma slug göndərilib	Postman-da Body **form-data** / **Text** qalıb — raw → JSON seç
`/api/categories/1/` → 404, amma kateqoriya var	URL-də `<int:category_id>`, funksiyada parametr adı fərqlidir
`TypeError: post_detail() got an unexpected keyword argument`	URL parametrinin adı ilə funksiya parametri uyğun deyil
Browsable API-də kateqoriya forması yoxdur	`@api_view`-da "POST" yoxdur — DRF formu yalnız POST varsa çəkir

`ProtectedError` (500) DELETE-də	Postu olan kateqoriya — normaldır (Dərs 10)
`AssertionError: You must call .is_valid()`	sıra səhvdir

Son 20 dəqiqə (2:40 – 3:00)

17 Yekun və ev tapşırığı

slayd 35

Slayd 35-i birlikdə oxu, `hw-14.pdf`-i payla. Push: `git add . → git commit -m "Dərs 14: API başlanğıc" → git push`. Yoxla: `git status`-da `.venv/` görünmür, `uv.lock` commit olunub.

> **De:** “Bu gün postlar üçün 2 funksiya yazdıq — 35 sətir. Siz kateqoriya üçün eynisini yazdınız — yenə 35. Evdə teq üçün — yenə 35. Gələn dərs bu 35 sətri **3 sətirə** salırıq: ViewSet + Router. Dərs 13-dəki fikir — hazır class, yalnız fərqi yaz. Və post JSON-unun içində şərhlər görünəcək.”

Ev tapşırığı **3 tapşırıq, ~25 dəqiqə** — hamısı bu gün yazılanın əkizidir (Tag ↔ Category, stats). Bunu de: “yeni heç nə yoxdur, kopyala-dəyiş.”

18 Ev tapşırığının həlli

yoxlamaq üçün

Tam həll bu qovluqdadır — `[EV 14]` nişanlı hissələr. Hər biri dərsdəki kodun əkizidir.

api/serializers.py — tapşırıq 1

```
~~from blog.models import Post, Category~~
from blog.models import Post, Category, Tag
...

# [EV 14] tapşırıq 1 — CategorySerializer-in əkizi
class TagSerializer(serializers.ModelSerializer):
    class Meta:
        model = Tag
        fields = ["id", "name", "slug"]
```

api/views.py — tapşırıq 1, 2, 3

```
# [EV 14] tapşırıq 1 — category_list-in əkizi
@api_view(["GET", "POST"])
def tag_list(request):
    if request.method == "GET":
        tags = Tag.objects.all()
        serializer = TagSerializer(tags, many=True)
        return Response(serializer.data)

    serializer = TagSerializer(data=request.data)
    if serializer.is_valid():
        serializer.save()
        return Response(serializer.data, status=status.HTTP_201_CREATED)
    return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)


# [EV 14] tapşırıq 2 — category_detail-in əkizi
@api_view(["GET", "PUT", "DELETE"])
def tag_detail(request, tag_id):
    tag = get_object_or_404(Tag, id=tag_id)

    if request.method == "GET":
        serializer = TagSerializer(tag)
        return Response(serializer.data)

    if request.method == "PUT":
        serializer = TagSerializer(tag, data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors, status=status.HTTP_400_BAD_REQUEST)

    tag.delete()
    return Response(status=status.HTTP_204_NO_CONTENT)
```

```python
# [EV 14] tapşırıq 3 — blog/views.py → stats səhifəsinin əkizi. Model yoxdur, serializer də lazım deyil: adi dict
@api_view(["GET"])
def stats(request):
    data = {
        "post_count": Post.objects.filter(is_published=True).count(),
        "category_count": Category.objects.count(),
        "tag_count": Tag.objects.count(),
    }
    return Response(data)
```

api/urls.py

```python
    # [EV 14]
    path("tags/", views.tag_list, name="tag_list"),
    path("tags/<int:tag_id>/", views.tag_detail, name="tag_detail"),
    path("stats/", views.stats, name="stats"),                    # tapşırıq 3
```

Bonus: repo-da `postman/` qovluğunda collection faylı (Postman ⋯ → **Export** → **v2.1**) — 6 sorğu.

> ⚠ Yoxlayarkən: `pyproject.toml` + `uv.lock` olmalıdır; `.venv/` GitHub-da olmamalıdır. `/api/tags/` 5 teq, `/api/stats/` 3 say. Müəllim üçün: `uv run manage.py test` — 23 test.